Lời dẫn seminar — Metropolis

24 slide chính; phần phụ lục ở PDF riêng để mở khi hỏi đáp.

Phần trình chiếu chính

Slide 01 — Điều khiển đèn giao thông từ dữ liệu

Mở đầu bằng vấn đề gần gũi: một hướng đang vắng vẫn được xanh, trong khi hướng kia có xe chờ. Dự án tạo môi trường mô phỏng để thử bốn bộ điều khiển trong cùng điều kiện. Học tăng cường sẽ được giải thích qua chính quyết định giữ xanh hoặc chuyển pha. Đây là nghiên cứu trong mô phỏng, chưa phải hệ thống đã triển khai ngoài đường.

Đối chiếu: Kho mã GAMA · benchmark đã hoàn tất

Slide 02 — Đèn nên giữ xanh hay nhường hướng khác?

Nêu đúng phần đóng góp của dự án. GAMA, IDM, MOBIL, Q-Learning và DQN là nền tảng kế thừa, không phải thuật toán do dự án phát minh. Phần triển khai là cấu hình và tích hợp mạng giao thông, luật bổ sung, điều khiển, kết nối Python và quy trình benchmark. Lỗi xe đứng vô cớ đã được xử lý bằng kiểm tra cùng làn con và tuyến đường thay cho chỉ kiểm tra khoảng cách hình học. Chuyển ý: Trước hết cần có một thế giới mô phỏng để thử quyết định này.

Đối chiếu: collision_avoidance.gaml; gama_env.py; fair_benchmark.py

Slide 03 — Môi trường: một ngã tư trong mạng nhiều tầng

Giải thích đây là mạng dựng bằng mã, không phải bản sao địa lý đã hiệu chỉnh của một nút giao thực. Mỗi nhánh mặt đất có đường vào và đường ra riêng. Các đường có thể cắt nhau trên hình chiếu nhưng cầu vượt và hầm không tạo giao lộ với mặt đất. Cao độ hiển thị cầu là +50 m và đáy hầm là −60 m theo cấu hình mô hình, không nên trình bày như số đo công trình thật. Chuyển động dùng topology phẳng và profile cao độ riêng. Đây là một tác tử điều khiển một nút giao, dù có bốn cột đèn. Trong phần hình ảnh, mỗi cột thể hiện thẳng và mũi tên trái. Không có bốn DQN độc lập. Xe rẽ phải và quay đầu tuân thủ tuyến và luật nhường/nhập dòng của mô hình; không nên suy ra mọi xe được tự do đi qua mọi xung đột.

Đối chiếu: collision_avoidance.gaml: init_doan_network; doan_scenery.gaml

Slide 04 — Mỗi làn đường có 16 làn con

Chỉ vào dãy từ dải phân cách tới lề phải. Chỉ số giảm từ 15 tới 0 do quy ước hiển thị của mô hình. Xe đi thẳng dùng vùng 4–11; ô tô chỉ được đặt chỉ số thấp nhất 4–10 để thân xe không vượt vùng. Buýt giữ chỉ số 8. Xe trái đặt ở 12, xe quay đầu ở 14; xe rộng chiếm cả ô kế bên. Các tuyến bypass giữ 16 slot logic để nối làn qua merge, nhưng phương tiện của từng tuyến bị khóa lane; không có nghĩa cầu vượt có 16 làn vật lý. Chuyển ý: Trên hạ tầng đó, xe được đưa vào và di chuyển như thế nào?

Đối chiếu: collision_avoidance.gaml: nb_lanes; spawn_vehicle_at; choose_lane

Slide 05 — Xe chạy theo luật

Giải thích ba phần: có xu hướng tăng tốc, giảm xu hướng đó khi gần tốc độ mong muốn, và trừ thêm phần phanh khi tiến sát xe trước. Không cần người nghe tính công thức ngay. Công thức khoảng cách

đầy đủ và đơn vị nằm ở phụ lục. Đã kiểm tra lớp IDM trong plugin GAMA cài trên máy: s sao dùng max(0, vT+vΔv/(2√ab)). Bộ quy tắc tổng thể trong dự án không phải IDM thuần túy vì còn các speed cap. Tác tử là một đối tượng có dữ liệu và luật hành động riêng; không phải mỗi xe có một mạng AI. Xe hiện chạy theo quy tắc giao thông và mô hình lái của GAMA. DQN chỉ học ở bộ điều khiển đèn. Hành vi ùn tắc xuất hiện từ tương tác nhiều xe. Nhịp 0,1 giây là thời gian trong thế giới mô phỏng; 800 giây giao thông có thể mất nhiều phút máy tính. Sơ đồ trình bày mô tả vòng lặp khái niệm, không cam kết thứ tự từng reflex nội bộ. Cần nói chính xác: skills: [driving] cung cấp kỹ năng, còn reflex move của dự án gọi do drive mỗi bước; không phải chỉ khai báo skill là xe tự chạy.

Đối chiếu: Treiber et al. (2000) [2]; GAMA [1]; IDM.class 2025.6.4

Slide 06 — Lách và rẽ trái đều có điều kiện

Lách không phải dịch ngang ngẫu nhiên. Code chỉ mở tối đa một lane kề bên trong allowed_lanes; GAMA sau đó áp dụng cơ chế đổi lane lấy cảm hứng từ MOBIL. Cần nói rõ hai tầng để không gán các ngưỡng 12 m, 1,5 giây hoặc 2,5 giây cho bài báo. Độ mượt nhìn thấy trên màn hình còn có nội suy hiển thị; nó không phải công thức học RL. Công thức cụ thể ở phụ lục. Công thức cap suy ra từ quan hệ động học $v^2=u^2+2as$, không phải nguyên văn mô hình Gipps. Luật cấp vào do dự án thiết kế: lối ra không có xe trong khoảng dài xe rẽ cộng 8 m. Busy approach được giữ khi xe vừa cam kết vào hoặc còn gần đầu connector trong khoảng dài xe cộng 12 m; vì vậy không nói cứng chỉ một xe trong toàn cung mọi lúc. Một xe khác có thể được cấp vào khi xe trước đã đi đủ xa. Các con số chưa hiệu chỉnh thực nghiệm. Công thức đa thức Bézier chuẩn theo bài giảng CS3621 của Michigan Technological University. Trong mã, doan_turn lấy 121 điểm t=0..1. Bốn điểm P0, P3 là đầu/cuối, P1/P2 định hình độ cong. Đây không phải công thức trích từ nghiên cứu RL. Chuyển ý: Xe đã có luật di chuyển. Bây giờ chuyển sang phần bộ đèn được biết và được làm.

Đối chiếu: MOBIL [3]; Michigan Tech CS3621 [9]; luật dự án

Slide 07 — AI được quyết định khi nào?

Bộ điều khiển không trực tiếp bật màu tùy ý cho từng bóng. GAMA giữ thứ tự và thời lượng các pha vàng/trái; Python trả về 0 hoặc 1. Sau chuyển pha phải qua 18 giây pha trung gian và 10 giây xanh tối thiểu hướng kia mới hỏi lại, cộng bước handshake. Vì vậy khoảng giữa quyết định thay đổi theo hành động. Lần chuyển sớm nhất thực tế khoảng 10,1 giây; Fixed-Time chuyển ở 30,1 giây với cùng giao diện. Fixed-Time là mốc cơ sở đã chọn với xanh bằng nhau cho hai trục, không phải chu kỳ đã tối ưu cho mọi nhu cầu. Không nên nói mọi đèn ngoài đường đều fixed. Max-Pressure phản ứng ngay theo công thức. Q-Learning và DQN học cách ước lượng lợi ích dài hạn. Độ phức tạp tăng không đảm bảo thứ hạng tốt hơn. Q rời rạc và DQN có biểu diễn khác nhau, nên “công bằng” ở đây là cùng demand, lịch đèn, reward và dữ liệu huấn luyện, không phải đầu vào được biến đổi hoàn toàn giống nhau.

Đối chiếu: intersection-dqn-python.gaml: dqn_controller, update_schedule

Slide 08 — Bốn phương pháp điều khiển

Chỉ đọc tên bốn phương pháp rồi chuyển sang slide Fixed-Time. Các slide tiếp theo mới giải thích cách mỗi phương pháp ra quyết định. Chuyển ý: Cùng hai lựa chọn ấy, bốn phương pháp sẽ quyết định khác nhau.

Đối chiếu: fair_benchmark.py: Learners.act; tabular_key; features

Slide 09 — Fixed-Time

Fixed-Time là mốc cơ sở đã chọn với xanh bằng nhau cho hai trục, không phải chu kỳ đã tối ưu cho mọi nhu cầu. Không nên nói mọi đèn ngoài đường đều fixed. Max-Pressure phản ứng ngay theo công thức. Q-Learning và DQN học cách ước lượng lợi ích dài hạn. Độ phức tạp tăng không đảm bảo thứ hạng tốt hơn. Q rời rạc và DQN có biểu diễn khác nhau, nên "công bằng" ở đây là cùng demand, lịch đèn, reward và dữ liệu huấn luyện, không phải đầu vào được biến đổi hoàn toàn giống nhau. Fixed-Time là mốc cơ sở: cùng thứ tự tám pha, xanh thẳng 30,1 giây; vàng 3 giây và trái 12 giây. Không có huấn luyện và không dùng số xe để chọn thời điểm. Đây là một cấu hình fixed-time cụ thể, không chứng minh mọi lịch cố định đều kém. Mốc 30,1 giây là 30 giây cộng bước handshake 0,1 giây của giao diện chung.

Đối chiếu: fair_benchmark.py: Learners.act; tabular_key; features

Slide 10 — Max-Pressure

Áp lực là độ dồn ứ ròng, không phải lực vật lý. Trừ xe phía thoát để tránh dồn thêm vào đầu ra đã đông. Dự án dùng biến thể hai trục với ngưỡng 2 và lịch pha ràng buộc. Max-pressure gốc xét các chuyển động, tỷ lệ rẽ và lưu lượng bão hòa; không bê nguyên định lý ổn định hay tối ưu của bài báo sang triển khai giản lược này. Ví dụ trên chỉ minh họa số học, không phải một dòng log đo được. Chuyển ý: Max-Pressure dùng một luật viết sẵn. Hai phương pháp tiếp theo học từ trải nghiệm.

Đối chiếu: Varaiya (2013) [4]; fair_benchmark.py: Learners.act

Slide 11 — Reinforcement Learning - Học tăng cường

Không cần ẩn dụ huấn luyện động vật. Dùng chính ngã tư để giải thích: bộ điều khiển thử giữ xanh, môi trường chạy thêm, hàng đợi tạo điểm phạt; từ đó điều chỉnh đánh giá của hành động. Không có nhãn sẵn "đây là hành động đúng" cho mọi trạng thái. Reward do người thiết kế đặt, còn cách chọn hành động được học. Vì quan sát bị gom và cắt phạm vi, state ở đây là vector quan sát, không phải trạng thái đầy đủ Markov đã được chứng minh.

Đối chiếu: Watkins & Dayan (1992) [5]; triển khai dự án

Slide 12 — Q-Learning: kinh nghiệm nằm trong bảng Q

Dùng một ví dụ về dòng dữ liệu: key là bộ sáu số mô tả tình huống; value là hai điểm cho giữ và chuyển. Số lớn hơn là tốt hơn, kể cả khi đều âm. Điểm Q ước lượng tổng điểm thưởng hiện tại và tương lai, không phải xác suất. Ví dụ (2,0,1,0,0,1) tương ứng [–3,–8] chỉ minh họa. Bảng trong Python là dictionary; khi lưu checkpoint PyTorch .pt, dictionary nằm ở trường table bên cạnh trạng thái mạng và optimizer. File best_q.pt là checkpoint được chọn bằng validation; khi test, script chỉ nạp trường table. Tối đa 216 key có thể có; các key chưa gặp không bắt buộc đã xuất hiện trong file. Công thức cập nhật là Q mới = Q cũ + alpha × [r + gamma × max Q(sau) − Q cũ], với alpha = 0,1; cuối episode không cộng phần tương lai.

Đối chiếu: scripts/fair_benchmark.py: tabular_key, Learners.act, Learners.learn, save, load_policy

Slide 13 — Sáu thành phần tạo nên một key của bảng Q

Đi từng hàng theo trục đang xanh và trục còn lại. Hai hàng đầu đo xe đi thẳng đang dừng theo ba mức: 0 xe, 1–4, từ 5 trở lên. Hai cờ tiếp theo báo có xe chờ rẽ trái hay không. Cờ lối ra bằng 1 khi lối ra của trục xanh có từ 5 xe trở lên. Tuổi xanh được chia dưới 25 giây, từ 25 tới dưới 45 giây, và từ 45 giây trở

lên. Nhân số lựa chọn của sáu thành phần: 3×3×2×2×2×3 = 216 tình huống tối đa; không phải có "sáu trạng thái". Với mỗi tình huống, bảng giữ hai điểm cho hai hành động.

Đối chiếu: scripts/fair_benchmark.py: tabular_key

Slide 14 — DQN: nhìn toàn bộ vòng ra quyết định và học

Sơ đồ đọc theo chiều trên: 11 số quan sát tại thời điểm hỏi đèn đi qua hai lớp ẩn, mỗi lớp có đúng 64 nút, rồi cho hai điểm Q. Chọn một hành động và GAMA chạy tiếp. Các chấm là nút, đường nối dày đặc giữa các nút đã lược bỏ; mỗi nút tính tổng có trọng số rồi qua ReLU. Sau đoạn mô phỏng mới biết reward r và quan sát s mới. Reward không phải một trong 11 đầu vào của lượt chọn vừa rồi. Bản ghi được đưa vào replay; khi đủ dữ liệu, lấy 64 bản ghi, lập mục tiêu bằng mạng target, tính loss, lan truyền gradient ngược qua mạng online, rồi Adam sửa W và b. Mạng target chỉ được sao chép sau mỗi 100 cập nhật. Trong benchmark, mỗi transition mới khi buffer đã đủ 64 mẫu dẫn tới bốn minibatch update.

Đối chiếu: scripts/fair_benchmark.py: Learners.learn; dqn_py/dqn_agent.py: QNetwork

Slide 15 — Mỗi nút của DQN tính gì?

Giải thích một nút trước khi nói cả mạng: lấy từng số từ lớp trước, nhân với trọng số tương ứng, cộng lại và thêm độ lệch b. Trọng số biểu thị mức ảnh hưởng mà mạng đang học, không phải quy tắc giao thông do ta viết sẵn. Hai lớp ẩn áp dụng ReLU: z âm cho ra 0, z dương được giữ. Ví dụ −1,2 thành 0 và 2,3 vẫn là 2,3. ReLU tạo tính phi tuyến, nhờ đó mạng biểu diễn quan hệ phức tạp hơn một phép nhân cộng đơn lẻ. Lớp đầu ra chỉ tuyến tính, vì hai điểm Q có thể âm; điểm Q không phải xác suất. 5.058 tham số gồm 11×64+64, 64×64+64 và 64×2+2. Khi học, backpropagation và Adam sửa các trọng số cùng độ lệch. DQN chỉ nhận vector 11 số quan sát; reward xe dừng toàn mạng dùng để học sau quyết định, không nhập vào mạng ở lượt suy luận đó.

Đối chiếu: Mnih et al. (2015) [6]; dqn_agent.py: QNetwork; fair_benchmark.py: Learners

Slide 16 — Từ 11 số đầu vào đến một lệnh đèn

Hai giá trị chỉ minh họa, không phải xác suất 4,2% hay 2,7%. Python gọi forward của mạng rồi argmax để lấy chỉ số điểm lớn nhất. Suy luận sử dụng torch.no_grad để không lưu đồ thị gradient. Với test, epsilon=0 và hoàn toàn không gọi learn. Trong train, actor có thể chọn ngẫu nhiên theo epsilon nên không luôn chọn Q lớn nhất. Không dùng 11 ảnh, 11 xe hay 11 cảm biến vật lý; đây là 11 số trong một vector. Hai cờ pha chỉ dùng để báo trục nào đang xanh ở lúc cần quyết định. Clipping có mất thông tin: 60 và 100 xe cùng thành 3. Q-table và DQN có cách biểu diễn khác nhau; bảng dùng mức rời rạc còn DQN nhận số đếm đã chuẩn hóa và chặn ngưỡng.

Đối chiếu: fair_benchmark.py: act, run_episode; gama_env.py: set_action

Slide 17 — Hàm thưởng: xe chờ lâu làm điểm giảm

Giải thích theo thời gian sau một lệnh đèn. Cứ 0,1 giây, GAMA đếm tất cả xe mặt đất trong mạng có tốc độ dưới 1 km/h. Script cộng số xe dừng × 0,1 giây qua toàn khoảng tới quyết định kế tiếp, rồi lấy âm và chia 50. Vì vậy 20 xe dừng đều 5 giây tạo 100 xe-giây, reward −2; 5 xe dừng đều 5 giây tạo 25 xe-giây, reward −0,5 và được ưa hơn. Giá trị này là điểm của người thiết kế, không phải tiền hay số xe. Hệ số 50 chỉ đổi thang số. Điểm phạt không dùng hàng xe còn ngoài mạng và chưa có TTC/PET. Thời gian giữa hai quyết định có thể khác nhau, nên tích phân theo từng bước thay vì chỉ nhìn hàng đợi tại cuối khoảng.

Đối chiếu: scripts/fair_benchmark.py: run_episode; Traffic/includes/collision_avoidance.gaml: queue_time_integral

Slide 18 — Quá trình train diễn ra như thế nào?

Nói theo một vòng làm việc: tạo 40 file nhu cầu từ seed 1001–1040; mỗi episode reset GAMA và chạy 8.000 bước × 0,1 giây. Actor DQN và Q-Learning thay phiên điều khiển. Trong nhóm tối đa bốn episode, mỗi actor dùng chính sách đã chụp ở đầu nhóm và thu transition. Sau nhóm, bộ học chính nhập transition theo thứ tự episode rồi cập nhật cả bảng Q lẫn mạng DQN. Cứ 10 episode, thử hai seed validation và lưu checkpoint có hàng đợi thấp nhất. Đã thu 2.381 transition, thực hiện 9.272 lần cập nhật DQN. Test dùng checkpoint đã chọn, không thử ngẫu nhiên và không học thêm. Một episode là một lượt mô phỏng, không phải một lần cập nhật.

Đối chiếu: scripts/fair_benchmark.py: pipeline, run_jobs, Learners.learn; training_log.json

Slide 19 — Thử hành động, lưu trải nghiệm, rồi học lại

Replay là đọc lại kinh nghiệm, không chạy lại toàn bộ giao thông cho mỗi mẫu. Mẫu ngẫu nhiên giúp giảm việc học chỉ theo chuỗi vừa diễn ra. Target network thay đổi chậm hơn online, giúp mục tiêu học bớt thay đổi liên tục; nó không phải nhãn thật từ một chuyên gia. Buffer của benchmark là deque chứa cả biểu diễn vector và khóa bảng Q. Mỗi transition mới cũng dùng để cập nhật Q-table. Trong một nhóm tối đa bốn lượt, actor là bản sao đóng băng của bộ học tại đầu nhóm. Thu thập kết thúc rồi master nhập dữ liệu theo thứ tự episode, tránh thứ tự tiến trình xong trước quyết định kết quả học. Actor không phải mạng riêng tự học liên tục từng bước. Epsilon(k)=max(0,05,1−0,95(k−1)/32), nên mức sàn bắt đầu ở episode 33, không phải chính xác trong 32 episode đầu như bản thô. Cùng dữ liệu chưa đồng nghĩa hai thuật toán dùng đúng cùng số phép tính.

Đối chiếu: Mnih et al. [6]; fair_benchmark.py: replay, learn

Slide 20 — Một lần backpropagation diễn ra thế nào?

Nhấn mạnh đây là một lần cập nhật tham số, không phải một episode. Mỗi lần lấy ngẫu nhiên 64 trải nghiệm. Online dự đoán Q(s,a) cho hành động đã chọn; target network tính y từ reward đã nhận và điểm tương lai của s mới. Nếu done=1, phần tương lai bằng 0. Tính Huber loss trung bình trên 64 mẫu. zero_grad xóa gradient của lượt trước; loss.backward dùng quy tắc dây chuyền để tính đạo hàm từ đầu ra về các lớp ẩn. Chặn chuẩn gradient ở 10 rồi Adam cập nhật chỉ W,b của online với learning rate 3×10^{-4}. Target không nhận gradient; cứ 100 update mới chép trọng số từ online. Một transition mới làm bốn lượt cập nhật như vậy sau khi buffer có ít nhất 64 bản ghi. Ví dụ một mẫu: Q=-3, r=-2, max Q(s mới)=-2,5, Δt=5 giây thì y=-4,425; dự đoán đang cao hơn mục tiêu. Gradient là độ nhạy loss đối với trọng số, không phải reward.

Đối chiếu: scripts/fair_benchmark.py: Learners.learn; PyTorch autograd

Slide 21 — Đánh giá bằng những lượt chưa dùng để học

Giải thích như học, chọn cách làm, rồi làm bài kiểm tra cuối. Validation lặp trên hai seed để chọn checkpoint nên không được gộp vào test. Chỉ có một seed khởi tạo bộ học là 42; 40 demand seed không có nghĩa 40 lần huấn luyện độc lập. Cấu hình nguồn và demand được ghi hash để phát hiện mã thay đổi. Bộ test có 20 lượt đã hoàn tất. Các baseline không có bước huấn luyện, nhưng dùng cùng simulator và action interface.

Đối chiếu: manifest.json; fair_benchmark.py: validate, pipeline; seminar_integrity_check.json

Slide 22 — Benchmark: bốn phương pháp, sáu chỉ số (1/2)

Bảng này và bảng sau là trung bình của 5 seed test ghép cặp. Xe hoàn thành là số xe mặt đất đã tới đích trước mốc 800 giây. Xe dừng trung bình lấy tích phân số xe dừng chia 800 giây. Chờ tích lũy trên mỗi yêu cầu cộng thời gian dừng trong mạng và hàng FIFO ngoài mạng, rồi chia 1.424 yêu cầu. DQN thấp nhất về hàng đợi trung bình trong bộ test này nhưng chênh với Q nhỏ. Slide cho xem trung bình để dễ đọc; báo cáo gốc có độ lệch chuẩn từng seed.

Đối chiếu: comparison_results/fair_benchmark_20261005_final/report.md; per_run_metrics.csv

Slide 23 — Benchmark: bốn phương pháp, sáu chỉ số (2/2)

Nhu cầu còn ngoài mạng ở cuối lượt khoảng 1.044–1.045 cho mỗi phương pháp; cổng nhận xe tại nguồn cũng hạn chế dòng vào. Hành trình chỉ tính xe đã hoàn thành, có thể bị thiên lệch khi số xe xong khác nhau. Chờ lớn nhất là thời gian dừng tích lũy lớn nhất trong mạng. Max-Pressure thấp nhất về hành trình xe đã xong và số yêu cầu còn ngoài mạng; DQN thấp nhất về chờ lớn nhất, nhưng độ lệch chuẩn của chỉ số đó cao. Không suy ra một thứ hạng phổ quát.

Đối chiếu: comparison_results/fair_benchmark_20261005_final/report.md; per_run_metrics.csv

Slide 24 — Ngoài ùn tắc, mô phỏng còn đo rủi ro

TTC dùng khoảng hở chia tốc độ tiếp cận tương đối cho cặp xe gần nhau; đếm mẫu dưới 1 giây. TIT tích lũy phần thiếu so với ngưỡng TTC qua từng bước 0,1 giây. PET đo khoảng thời gian từ xe trước rời vùng xung đột tới xe sau vào vùng đó; đếm mẫu dưới 1,5 giây. DRAC lấy bình phương tốc độ tương đối chia hai lần khoảng hở, với chặn khoảng hở 0,5 m và trần 15 m/s². Phanh gấp là mẫu gia tốc dưới −3,4 m/s²; dưới −5,0 m/s² là phanh rất mạnh. Các bộ đếm TTC và phanh gấp tăng theo bước mô phỏng, không phải số sự cố độc lập. Mã có SAFETY_CSV, nhưng benchmark công bằng vừa chạy chưa đưa chúng vào metrics.json/per_run_metrics.csv. Chưa thể tuyên bố thứ hạng an toàn của bốn phương pháp.

Đối chiếu: collision_avoidance.gaml (đo TTC, PET, TIT, DRAC, phanh); fair_benchmark.py (chỉ số báo cáo)

Ghi chú bổ sung cho các slide đã bỏ

CPU tính, RAM giữ dữ liệu, ổ đĩa lưu mô hình

Trả lời trực tiếp “xử lý ở RAM hay gì”: CPU thực thi phép tính, RAM cung cấp chỗ chứa dữ liệu cho CPU, ổ đĩa lưu lâu dài. Pipeline này không chuyển tensor sang CUDA/GPU. 5.058 số float32 ×4 byte =20.232 byte cho tham số một mạng; còn target, actor snapshots, gradients, Adam, Python và JVM nên không được gọi 20 kB là toàn bộ bộ nhớ sử dụng. Lệnh chạy đặt _JAVA_OPTIONS Xmx2g, ghi đè giá trị 16G trong launcher; bản thô ghi 16 GB mỗi JVM là sai cho lần benchmark này. File .pt là checkpoint PyTorch, không phải video hoặc ảnh mô phỏng. Huấn luyện cần online, target, optimizer, replay và RNG để resume nhất quán. Đánh giá chỉ cần phần chính sách: online của DQN hoặc bảng Q. Pipeline lưu sau mỗi nhóm episode; nó không lưu toàn bộ trạng thái GAMA đang chạy giữa một episode. Một lần lỗi đóng cổng đã được khắc phục rồi resume từ checkpoint sau episode 4; phần dữ liệu học không bị mất. Chuyển ý: Máy đã tính được điểm Q; câu hỏi tiếp theo là lấy tiêu chí nào để dạy nó?

Đối chiếu: fair_benchmark.py: torch.set_num_threads, Learners; lifecycle/run configuration

Seed là gì? Bốn cách đèn cùng một lịch xe

Ví dụ seed 5001: thời điểm yêu cầu xe ở mỗi nguồn là định kỳ, còn Python dùng random.Random(5001) để lấy loại xe, tốc độ, hướng rẽ và làn bắt đầu. Các dòng được ghi vào demand/5001.csv; một dòng có thời điểm, nguồn, loại xe, tốc độ, hướng rẽ và lane. Script có thể sinh lại file khi lập job, nhưng cùng seed luôn tạo đúng cùng nội dung byte. Mỗi phương pháp được kiểm tra trên lịch yêu cầu giống hệt, nên khác biệt kết quả không do may mắn nhận một danh sách xe dễ hơn. Seed 5002 tạo tình huống khác và lại dùng chung cho cả bốn phương pháp. Manifest ghi danh sách seed; metrics từng lượt ghi SHA-256 của CSV để đối chiếu. Cùng lịch yêu cầu không đồng nghĩa xe thực sự được nhận vào mạng cùng thời điểm, vì cổng nguồn có thể bị chặn bởi ùn tắc.

Đối chiếu: fair_benchmark.py (tạo lịch xe và lưu kết quả); manifest.json

DQN giảm hàng đợi trong bộ kiểm thử này

Đây là số xe dừng trung bình theo thời gian của toàn mạng mặt đất, không phải độ dài hàng mét và không phải chỉ xe chờ đèn đỏ. Trong tập này thứ tự trung bình phù hợp giả thuyết nhưng không được trình bày như quy luật mọi nơi. DQN giảm khoảng 47,7% so với Fixed-Time. Chênh DQN với Q-Learning chỉ khoảng 0,58 xe; phải nhìn kết quả từng seed và sự biến thiên.

Đối chiếu: test_results.json: 20 lượt, 8.000 cycle/lượt

Kết quả tốt hơn có giới hạn

Mỗi chỉ số trả lời một câu khác nhau. Hành trình chỉ tính xe đã hoàn thành nên có thiên lệch chọn mẫu nếu một chính sách để xe chậm chưa kịp xong. Chờ/yêu cầu cộng thời gian dừng trong mạng và thời gian FIFO bên ngoài, rồi chia 1.424 yêu cầu; không phải thời gian chờ trọn đời của mọi chuyến. Chênh lệch này nhỏ hơn mức cải thiện chỉ số trong mạng do phần ngoài mạng rất lớn. Đây là các seed ghép cặp: cùng cột seed nhận cùng demand. DQN thắng 5001,5002,5004 nhưng kém nhất 5003; Q-Learning có độ lệch chuẩn queue nhỏ nhất trong bộ test. Không lấy việc chênh nhỏ hơn độ lệch chuẩn làm kiểm định thống kê. Nói rằng chưa có bằng chứng thống kê vững về ưu thế DQN so với Q; cần nhiều training seed và nhiều kịch bản. Chuyển ý: Kết lại, đóng góp của dự án nằm ở chuỗi triển khai và kiểm chứng này.

Đối chiếu: summary.csv; per_run_metrics.csv

Phụ lục hỏi đáp

Phụ lục 01 — A01 · Thứ tự 8 pha được giữ cố định

Đọc theo trật tự: thẳng Bắc–Nam, vàng, trái Bắc–Nam, vàng, rồi lặp tương tự Đông–Tây. Bộ học không chọn tùy ý tám pha mà chỉ giữ hay kết thúc xanh thẳng. Lưu ý quan trọng từ mã nguồn: vàng kết thúc sau 3 giây, không chờ junction_clear; hiện không có pha all-red riêng. Vì vậy chỉ gọi đây là ràng buộc lịch đèn của mô hình, không tuyên bố đã chứng minh an toàn vận hành thực tế.

Đối chiếu: intersection-dqn-python.gaml: dqn_controller

Phụ lục 02 — A02 · 11 đầu vào của mạng được tạo như thế nào?

Không dùng 11 ảnh, 11 xe hay 11 cảm biến vật lý; đây là 11 số trong một vector. Hai cờ pha chỉ dùng để báo trục nào đang xanh ở lúc cần quyết định. Clipping có mất thông tin: 60 và 100 xe cùng thành 3.

Q-table và DQN có cách biểu diễn khác nhau; bảng dùng mức rời rạc còn DQN nhận số đếm đã chuẩn hóa và chặn ngưỡng.

Đối chiếu: fair_benchmark.py: EXTRA_FIELDS, features

Phụ lục 03 — A03 · Huấn luyện có cải thiện trên validation

Đường này không phải loss train và không phải kết quả test. Nó đo hiệu quả giao thông khi đóng băng mô hình sau mỗi 10 episode. Trình bày rằng hiệu quả có thể lên xuống; số update tăng không buộc hàng đợi giảm đều. Q-Learning chọn checkpoint episode 40 (12,30 xe); DQN chọn episode 30 (9,61 xe), vì DQN episode 40 tăng lên 11,54 xe. Bốn điểm validation chưa chứng minh hội tụ.

Đối chiếu: validation/ep10…ep40; biểu đồ sinh trực tiếp từ metrics.json

Phụ lục 04 — A04 · Một phương pháp không thắng ở mọi chỉ số

Mỗi chỉ số trả lời một câu khác nhau. Hành trình chỉ tính xe đã hoàn thành nên có thiên lệch chọn mẫu nếu một chính sách để xe chậm chưa kịp xong. Chờ/yêu cầu cộng thời gian dừng trong mạng và thời gian FIFO bên ngoài, rồi chia 1.424 yêu cầu; không phải thời gian chờ trọn đời của mọi chuyến. Chênh lệch này nhỏ hơn mức cải thiện chỉ số trong mạng do phần ngoài mạng rất lớn.

Đối chiếu: summary.csv; per_run_metrics.csv

Phụ lục 05 — A05 · IDM đầy đủ và các đại lượng

Công thức được đối chiếu bytecode IDM.class của GAMA 2025.6.4 cài tại máy, gồm max(0,…) của khoảng cách động. T và delta không được dự án gán riêng trong configure_vehicle nên kế thừa mặc định skill; không tự điền số không kiểm tra. Các thông số xe riêng ở bảng kế tiếp.

Đối chiếu: [1], [2]; IDM.class; configure_vehicle

Phụ lục 06 — A06 · Cấu hình ba loại xe

Phân biệt nguồn mô hình với nguồn tham số. IDM có bài báo nền tảng, nhưng không vì vậy mà các số trong configure_vehicle tự trở thành tham số xác thực tại Việt Nam. max_deceleration trong skill được dùng như b của IDM và ở các cap tự viết; không nên diễn giải đơn giản như giới hạn vật lý tuyệt đối mà mọi phương trình luôn tuân thủ.

Đối chiếu: collision_avoidance.gaml: configure_vehicle; generate_demand

Phụ lục 07 — A07 · MOBIL và bộ lọc khoảng trống

Công thức đầu là mô hình tham khảo kinh điển; không khẳng định tất cả chi tiết của MOBIL nguyên bản đều có trong plugin. Trong project allowed_lanes thường chỉ chứa lane hiện tại. Bộ lọc mở một lane kề khi vượt điều kiện. Gần nút, điều kiện dùng khoảng cách từ tâm > distance_to_intersection_center+13 =53,5 m; với vạch dừng ở45 m, không nên nói chính xác “13 m trước vạch” như bản thô. Gần merge bị khóa khi distance_to_current_target <120 m.

Đối chiếu: Kesting et al. [3]; GAMA [1]; choose_safe_lane

Phụ lục 08 — A08 · Hình học và luật tốc độ tự triển khai

Không dùng công thức speed cap như lời hứa không va chạm trong mọi điều kiện. Nó kết hợp với driving skill và phản ứng theo bước. Việc chặn nhầm xe khác làn đã được sửa bằng xét chồng lấn khoảng lane thấp nhất tới thấp nhất+num_lanes_occupied−1. Hình rẽ phải trong bản thô ghi 15 km/h như mức chung là thiếu chính xác: cap 15 chỉ kích hoạt khi xe vào phạm vi tâm hẹp; road_kind right có maxspeed20 km/h. Vì vậy phần chính không dùng một số tuyệt đối cho mọi cung phải.

Đối chiếu: doan_turn; speed caps; Bézier [9]; động học [11]

Phụ lục 09 — A09 · Trạng thái bảng Q và phạm vi đếm xe

Công bằng phải được mô tả có phạm vi. Hai bộ học được dùng cùng dữ liệu chuyển tiếp nhưng có hàm đặc trưng khác nhau và update rule khác. Đây là so sánh hai cấu hình cụ thể, không phép thử chỉ thay đúng một yếu tố "có mạng hay không". Trạng thái chưa có trong bảng được xem là [0,0], tie argmax chọn hành động 0; chưa gặp không có nghĩa chương trình không chạy được.

Đối chiếu: tabular_key; features; observe_benchmark_movements

Phụ lục 10 — A10 · Loss, gradient và Adam

Phần chính chỉ cần hiểu loss là sai lệch, gradient là hướng sửa và Adam là quy tắc sửa. Huber gần 0 giống bình phương, khi sai số lớn tăng tuyến tính để giảm ảnh hưởng mẫu cực trị. Gradient clipping giới hạn chuẩn vector gradient, không giới hạn mỗi trọng số nằm trong [–10,10]. Không gọi cập nhật tham số bằng gradient là công thức Adam chính xác.

Đối chiếu: Kingma & Ba [8]; fair_benchmark.py: learn; PyTorch Adam/SmoothL1Loss [10]

Phụ lục 11 — A11 · Từ điển thuật ngữ để trả lời hỏi đáp

Kết hợp bảng này với slide định nghĩa step, decision, transition, episode, update và slide epsilon. Trong replay, một transition có thể được học nhiều lần. Trong batch rollout, actor là snapshot của chính sách. "Epoch" thường chỉ một lượt qua dataset cố định; ở pipeline này không dùng epoch làm đơn vị ngân sách.

Đối chiếu: Thuật ngữ theo triển khai fair_benchmark.py; [5], [6], [8]

Phụ lục 12 — A12 · Nguồn mô hình giao thông

Các đường liên kết mở tài liệu gốc hoặc tài liệu chính thức. Bài GAMA có ví dụ hiệu chỉnh ở Việt Nam nhưng dự án hiện tại không tự động kế thừa kết quả xác thực của bài báo. Chỉ nói kế thừa framework và cơ chế, không nói đã kiểm định bản đồ này bằng dữ liệu đó.

Đối chiếu: Nguồn gốc/nhà xuất bản; truy cập 06/10/2026

Phụ lục 13 — A13 · Nguồn học tăng cường và triển khai

Các công thức reward, số lớp 64–64, epsilon, gamma theo thời gian và lịch cập nhật là cấu hình của dự án. Mnih giới thiệu cơ chế DQN; không lấy mạng Atari hoặc hàm thưởng Atari để mô tả mô hình đèn. Dữ liệu benchmark là bằng chứng tại kho mã, không phải từ các bài tham khảo này.

Đối chiếu: Nguồn gốc và tài liệu chính thức; truy cập 06/10/2026